SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO2_AT1 Scenario-Based Assessment

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

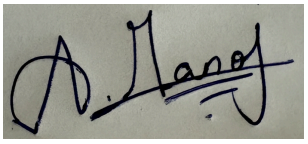
Assessment Tool Weightage: 50%

Code of Conduct:

I Manoj A / 192421295 that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question and Answers

1. An online banking system fails frequently during fund transfers at the minimum and maximum transfer limits. Analyse the current test design, apply Equivalence Partitioning and Boundary Value Analysis, assess random testing, and propose an optimised approach.

Answer:

The failures at the minimum and maximum limits show that the existing test cases are concentrated on typical amounts such as ₹500 or ₹5,000 and rarely touch the edges of the valid range. Programming mistakes are most common exactly there, for example writing amount < 50000 instead of amount <= 50000. Random testing gives no guarantee that these edge values are ever selected, so the same defects reach production again and again.

i) Analysis of the current test strategy

- **No traceability:** test inputs are not derived from the requirement (valid range ₹1 to ₹50,000), so nobody can say which rules were verified.
- **Unpredictable coverage:** the chance that a random value hits exactly ₹1 or ₹50,000 is extremely small.
- **Not repeatable:** a random run cannot be reproduced for retesting or regression after a fix.
- **Missing negative cases:** zero, negative, decimal, blank and non-numeric amounts, and amounts above the account balance are rarely tried.

ii) Equivalence Partitioning (ECP)

The amount field is divided into classes in which every value is expected to be treated the same way by the program. One representative value per class is enough.

- **EC1 (valid)** – Description: ₹1 to ₹50,000 and balance sufficient; Representative value: ₹5,000; Expected result: Transfer successful
- **EC2 (invalid)** – Description: Less than ₹1 (zero or negative); Representative value: ₹0 / ₹-100; Expected result: Error: invalid amount
- **EC3 (invalid)** – Description: Greater than ₹50,000; Representative value: ₹60,000; Expected result: Error: limit exceeded
- **EC4 (invalid)** – Description: Valid amount but more than balance; Representative value: ₹30,000 with ₹10,000 balance; Expected result: Error: insufficient funds
- **EC5 (invalid)** – Description: Non-numeric, blank or more than 2 decimals; Representative value: abc / blank / 100.555; Expected result: Error: invalid format

iii) Boundary Value Analysis (BVA)

For the range ₹1 to ₹50,000 the values at and next to each boundary are selected, because defects cluster at the edges.

- **BV1** – Input (₹): 0; Boundary position: Just below minimum; Expected result: Rejected
- **BV2** – Input (₹): 1; Boundary position: Minimum; Expected result: Accepted
- **BV3** – Input (₹): 2; Boundary position: Just above minimum; Expected result: Accepted
- **BV4** – Input (₹): 49,999; Boundary position: Just below maximum; Expected result: Accepted
- **BV5** – Input (₹): 50,000; Boundary position: Maximum; Expected result: Accepted
- **BV6** – Input (₹): 50,001; Boundary position: Just above maximum; Expected result: Rejected

iv) Should random testing be retained or replaced?

Random testing should be replaced as the primary technique, because it cannot provide measurable coverage or guaranteed boundary checking. It can still be kept in a small, controlled role: exploratory sessions of limited time after the structured tests are completed, to discover unexpected behaviour such as double-clicking the transfer button or switching networks during the transaction. In that role it supplements the structured design; it does not substitute for it.

v) Proposed structured and optimised approach

- Derive every test case from the requirement and record it in a Requirement Traceability Matrix.
- Apply ECP first to find the classes, then BVA on every numeric boundary (amount, daily limit, OTP attempts).
- Combine the amount tests with other conditions such as account type, balance, payee type and daily cumulative limit using a decision table.
- Automate the BVA and ECP tests as a regression suite that runs after every build.
- Use risk-based prioritisation so that money-related transfers are tested before cosmetic features.
- Add every production defect as a permanent regression test, and review test cases regularly to avoid the pesticide paradox.

vi) Justification

The approach reduces the number of tests from practically unlimited random inputs to about a dozen well-chosen cases while covering every class and every boundary. Defect detection becomes systematic and measurable, tests are reproducible, and the team gains confidence

that the limits work. The structured design therefore finds boundary defects earlier, lowers production failures, and protects customers' money.

2. A healthcare application module was tested but critical defects appeared after deployment because of insufficient coverage. Construct a conceptual control flow graph, assess statement, branch and path coverage, and design an improved white-box strategy.

Answer:

The post-deployment defects show that the test suite verified only the normal flow of the module and left several decisions and paths unexecuted. In a healthcare system this is critical because an untested path may be a wrong dosage or a missed allergy alert. White-box testing uses the structure of the code to judge test adequacy.

i) Conceptual module (medication dose check)

- N1: Read age, weight, allergy flag and prescribed dose.
- D1: If age ≤ 0 or weight ≤ 0, go to N2 (reject: invalid patient data).
- D2: If the patient is allergic to the drug, go to N3 (block prescription and alert).
- D3: If dose > maximum dose, go to N4 (reject: overdose).
- D4: If dose < minimum dose, go to N5 (warn: under-dose), otherwise go to N6 (accept prescription).
- N7: Write audit log and end.

ii) Control flow graph and cyclomatic complexity

The graph has four decision nodes (D1 to D4) and seven statement nodes. Cyclomatic complexity V(G) = number of decisions + 1 = 4 + 1 = 5, so five independent paths must be tested at minimum.

- **P1** – Route through the graph: N1 → D1(T) → N2 → N7; Meaning: Invalid patient data
- **P2** – Route through the graph: N1 → D1(F) → D2(T) → N3 → N7; Meaning: Allergy block
- **P3** – Route through the graph: N1 → D1(F) → D2(F) → D3(T) → N4 → N7; Meaning: Overdose rejected
- **P4** – Route through the graph: N1 → D1(F) → D2(F) → D3(F) → D4(T) → N5 → N7; Meaning: Under-dose warning
- **P5** – Route through the graph: N1 → D1(F) → D2(F) → D3(F) → D4(F) → N6 → N7; Meaning: Normal accepted prescription

iii) Gaps in the existing testing

Suppose the earlier suite contained only two tests: one normal prescription (P5) and one invalid patient record (P1). The coverage achieved can be assessed as follows.

- **Statement coverage** – Calculation: Executed N1, N2, N6, N7 = 4 of 7 statements; Result: 57%
- **Branch coverage** – Calculation: Executed D1-T, D1-F, D2-F, D3-F, D4-F = 5 of 8 branches; Result: 62.5%
- **Path coverage** – Calculation: 2 of 5 independent paths; Result: 40%

The allergy block, overdose rejection and under-dose warning logic were never executed. These are exactly the safety-critical branches, so defects in them were discovered only in production. The suite also did not meet any recognised adequacy criterion such as 100% branch coverage.

iv) Improved testing strategy

- Design one test for each of the five basis paths, which gives 100% statement, 100% branch and 100% basis-path coverage with five test cases.
- Add boundary tests around each decision: age 0 and 1, weight 0 and 0.1, dose equal to maximum, maximum + 1, minimum and minimum − 1.
- Use a coverage tool (JaCoCo, Coverage.py or gcov) to measure coverage automatically and block a release when branch coverage is below the agreed target (for example 100% for safety-critical modules).
- Add condition coverage for compound decisions such as age ≤ 0 OR weight ≤ 0, so each condition is tested both true and false.
- Perform code review of the CFG to confirm that no unreachable code or missing else-branch exists.

v) How this improves reliability and reduces risk

Every statement and every decision outcome is now executed at least once, so hidden logic errors are found before deployment. Test adequacy becomes a measured number rather than an opinion, which makes release decisions defensible. For a healthcare module this directly reduces patient-safety risk, regulatory risk and the cost of emergency fixes.

3. A retail e-commerce system has several input fields (price, quantity, discount), but testers missed errors in certain input combinations. Evaluate the current black-box approach and propose a revised design using ECP, BVA and decision table testing.

Answer:

The testers verified each field mostly in isolation, with one or two typical values. Many defects, however, appear only when fields interact, for example a valid discount applied to a

valid quantity that produces a total below zero. The current approach is limited because it ignores combinations, boundaries and invalid inputs.

i) Limitations of the current approach

- Fields are tested individually, so interaction defects remain hidden.
- Only normal values are used, so boundary and invalid values are not checked.
- Testing every combination is impractical: if each of the three fields has just 8 useful values, there are $8 \times 8 \times 8 = 512$ combinations.
- There is no documented rule table, so the expected result for a combination is decided informally.

ii) Equivalence classes (assumed rules)

Assume price ₹1 to ₹1,00,000, quantity 1 to 100, and discount 0% to 50%.

- **Price** – Valid class: $1 \leq$ price $\leq$ 1,00,000; Invalid classes: price ≤ 0; price $>$ 1,00,000; non-numeric
- **Quantity** – Valid class: $1 \leq$ qty ≤ 100; Invalid classes: qty ≤ 0; qty > 100; decimal; blank
- **Discount** – Valid class: $0 \leq$ discount ≤ 50; Invalid classes: discount < 0; discount > 50; non-numeric

iii) Boundary value analysis

- **Price** – Boundary test values: 0, 1, 2, 99,999, 1,00,000, 1,00,001
- **Quantity** – Boundary test values: 0, 1, 2, 99, 100, 101
- **Discount** – Boundary test values: -1, 0, 1, 49, 50, 51

iv) Decision table testing for combinations

The conditions are C1 = price valid, C2 = quantity valid and C3 = discount valid. The action is to calculate the total or show an error.

- **R1** – C1 Price: Valid; C2 Qty: Valid; C3 Discount: Valid; Expected action: Calculate total = price × qty × (1 − discount)
- **R2** – C1 Price: Valid; C2 Qty: Valid; C3 Discount: Invalid; Expected action: Error: invalid discount
- **R3** – C1 Price: Valid; C2 Qty: Invalid; C3 Discount: Valid; Expected action: Error: invalid quantity
- **R4** – C1 Price: Valid; C2 Qty: Invalid; C3 Discount: Invalid; Expected action: Error: invalid quantity and discount
- **R5** – C1 Price: Invalid; C2 Qty: Valid; C3 Discount: Valid; Expected action: Error: invalid price

- **R6** – C1 Price: Invalid; C2 Qty: Valid; C3 Discount: Invalid; Expected action: Error: invalid price and discount
- **R7** – C1 Price: Invalid; C2 Qty: Invalid; C3 Discount: Valid; Expected action: Error: invalid price and quantity
- **R8** – C1 Price: Invalid; C2 Qty: Invalid; C3 Discount: Invalid; Expected action: Error: all fields invalid

v) Revised test design strategy

- Use ECP and BVA to select a small set of values for each field instead of all possible values.
- Use the decision table to cover every valid and invalid combination (8 rules), with at least one test for each rule.
- Add pairwise (all-pairs) testing for extra attributes such as customer type, coupon and payment mode; this reduces hundreds of combinations to about 10 to 15 tests while still covering every pair of values.
- Add calculation checks at the extremes, such as ₹1,00,000 × 100 at 50% discount, to detect overflow and rounding errors.
- Verify the total independently by calculating the expected value manually or with a spreadsheet.

vi) Justification

ECP removes redundant tests, BVA targets the places where defects cluster, and the decision table guarantees that no combination is forgotten. Compared with isolated testing, the revised strategy increases defect detection for interaction errors, gives clear completeness criteria, and keeps the test count manageable.

4. A module with complex nested conditions and loops shows only partial path coverage. Evaluate the white-box approach, identify gaps in CFG construction and path testing, and propose a complete path testing strategy.

Answer:

Partial path coverage means that the team did not identify all independent paths through the module. This usually happens when the control flow graph is drawn informally, loop exits are missed, or a compound condition is treated as one decision. The unexecuted paths then hide defects.

i) Conceptual module

- 1: total = 0, i = 0
- 2: while (i < n) → loop decision (D1)
- 3: if (item[i].qty > 0) → D2

- 4: if (item[i].price > 1000) → D3: apply 10% discount, else add normal price
- 5: i = i + 1 and return to the loop
- 6: if (total > 5000) → D4: set free shipping
- 7: return total

ii) Gaps in the current approach

- The CFG probably omitted the loop-back edge and the zero-iteration path (n = 0).
- Nested decisions D2 and D3 were treated as one block, so the combination (qty = 0) and (qty > 0, price ≤ 1000) was never separated.
- Only complete paths with one or two loop iterations were tried; the number of possible paths is unbounded when loops exist, so a rule for choosing paths was missing.
- Independent paths were not counted using cyclomatic complexity, so the team had no target number.

iii) Impact on defect detection

Every missed independent path is a combination of decisions never executed. For example, if the discount path is never run together with the free-shipping condition, an error in the discount calculation that pushes the total above or below ₹5,000 remains hidden. Missing the zero-iteration path may hide a crash on empty input. Because defects in nested logic often depend on a specific order of decisions, partial path coverage gives a false sense of security.

iv) Cyclomatic complexity and basis paths

Predicate nodes are D1 (loop), D2, D3 and D4, so V(G) = 4 + 1 = 5. The basis set therefore contains five independent paths.

- **P1** – Description: Loop not entered (n = 0), total ≤ 5000; Sample test: n = 0
- **P2** – Description: Loop once, qty = 0 (item skipped), total ≤ 5000; Sample test: qty = 0
- **P3** – Description: Loop once, qty > 0, price ≤ 1000, total ≤ 5000; Sample test: qty = 1, price = 500
- **P4** – Description: Loop once, qty > 0, price > 1000 (discount), total ≤ 5000; Sample test: qty = 1, price = 2000
- **P5** – Description: Loop once, qty > 0, total > 5000 (free shipping); Sample test: qty = 1, price = 6000

v) Complete path testing strategy

- Draw the CFG with separate nodes for every decision and for every loop-back and loop-exit edge.
- Compute V(G) and design at least one test for every basis path (five tests above).
- Add loop-boundary tests: zero iterations, one iteration, two iterations, a typical number, the maximum n, and maximum + 1.

- Add condition tests for decision D4 around the boundary: total = 4,999, 5,000 and 5,001.
- Measure statement, branch and path coverage with a coverage tool and continue until all basis paths and 100% branch coverage are achieved.
- Prioritise the critical paths (calculation and shipping) for extra data-flow checks.

vi) Expected outcomes

The five basis paths give 100% statement and branch coverage and guarantee that each decision outcome is executed at least once. Loop-boundary tests expose off-by-one and empty-input errors. The result is a documented, measurable path testing process, fewer residual defects, and better confidence in the nested logic.

5. A financial application uses both black-box and white-box testing, yet defects still reach production. Analyse how the techniques were integrated, check whether adequacy criteria were applied, and propose an improved hybrid strategy.

Answer:

The presence of both techniques does not by itself ensure quality. Defects continue to reach production when the two techniques are applied separately, without common objectives, measurable adequacy criteria, or traceability between requirements, test cases and code.

i) Analysis of the current integration

- Black-box tests (ECP, BVA) were probably designed from requirements, while white-box tests were written independently by developers; nobody compared what each set covered.
- No test adequacy criteria were defined, such as the required percentage of branch coverage or the requirement coverage target.
- Coverage metrics were collected, if at all, only for unit tests and not for the combined suite.
- Requirements that were missing from the code could not be found by white-box testing, while code paths that the requirements did not mention could not be found by black-box testing.
- There was no risk ranking, so interest calculation and transaction logic received the same effort as low-risk screens.

ii) Were adequacy criteria and metrics applied properly?

The evidence suggests they were not. Adequacy criteria define when testing is sufficient, for example 100% requirement coverage, all boundaries tested, and 90% to 100% branch coverage on critical modules. Without them, the team stopped when the schedule ended, not when the evidence was enough.

iii) Proposed hybrid strategy

- **Requirements** – Technique: Traceability matrix; Purpose: Every requirement has tests; Adequacy criterion: 100% requirement coverage
- **Inputs** – Technique: ECP and BVA; Purpose: Find input classes and boundary errors; Adequacy criterion: All classes and boundaries covered
- **Rules** – Technique: Decision table; Purpose: Verify business-rule combinations; Adequacy criterion: Every rule tested
- **Code structure** – Technique: CFG and basis path testing; Purpose: Execute each independent path; Adequacy criterion: All basis paths, 100% branch on critical code
- **Conditions** – Technique: Condition and MC/DC coverage; Purpose: Verify compound conditions; Adequacy criterion: MC/DC for calculation logic
- **Quality of tests** – Technique: Mutation testing; Purpose: Check that tests detect injected faults; Adequacy criterion: Mutation score above 80%

iv) Working process

- Step 1: Design black-box tests from requirements using ECP, BVA and decision tables.
- Step 2: Execute them and measure code coverage.
- Step 3: Examine uncovered statements and branches; draw the CFG for those parts and add white-box tests for the remaining independent paths.
- Step 4: Use the same defect repository to analyse escaped defects and add them to regression tests.
- Step 5: Make the release conditional on meeting the adequacy criteria and on no open critical defects.

v) Example

Consider a loan-interest function. BVA tests check loan amounts at the minimum and maximum limits and tenure at 1 and 360 months. Coverage measurement then shows that the branch for a premature-closure penalty was never executed. A white-box test is added for that path, which reveals a wrong penalty formula that black-box tests had missed.

vi) Justification and conclusion

Black-box testing verifies what the system should do, white-box testing verifies how the code does it, and measurable criteria show when enough testing has been done. Combining them with traceability and mutation testing closes the gaps that each technique has alone. The hybrid approach therefore increases coverage, detects defects before release, and prevents recurrence through regression tests.

6. An HR payroll system gives incorrect bonuses to employees with exactly 5, 10 and 15 years of service. Analyse the failure, evaluate the test design, apply BVA and ECP, and justify these techniques over exploratory testing.

Answer:

The incorrect payouts at exactly 5, 10 and 15 years point to a boundary defect. The bonus bands are probably written with the wrong relational operator, for example years > 5 instead of years >= 5, so the employee who has exactly completed the period is placed in the lower band. This is a classic off-by-one error.

i) Analysis of the failure and the test design

Assume the business rule: 0 to 4 years = 0% bonus, 5 to 9 years = 5%, 10 to 14 years = 10%, and 15 years or more = 15%, multiplied by a performance rating factor from 1 (poor) to 5 (outstanding). The testers probably used values such as 2, 7, 12 and 20 years, which lie inside the bands and therefore pass even though the comparison operators are wrong. Boundary values were never tested, and rating combinations were not systematically varied.

ii) Equivalence class partitioning

- **EC1** – Years of service: 0 to 4; Bonus band: 0%; Representative value: 2
- **EC2** – Years of service: 5 to 9; Bonus band: 5%; Representative value: 7
- **EC3** – Years of service: 10 to 14; Bonus band: 10%; Representative value: 12
- **EC4** – Years of service: 15 or more; Bonus band: 15%; Representative value: 20
- **EC5 (invalid)** – Years of service: Negative; Bonus band: Error; Representative value: -1
- **EC6 (invalid)** – Years of service: Non-numeric or blank; Bonus band: Error; Representative value: abc / blank

Performance rating is also partitioned: valid ratings 1 to 5 and invalid ratings 0, 6, negative and non-numeric.

iii) Boundary value analysis

- **B1** – Years: 4; Boundary: Upper limit of band 1; Expected bonus band: 0%
- **B2** – Years: 5; Boundary: Lower limit of band 2; Expected bonus band: 5%
- **B3** – Years: 6; Boundary: Just inside band 2; Expected bonus band: 5%
- **B4** – Years: 9; Boundary: Upper limit of band 2; Expected bonus band: 5%
- **B5** – Years: 10; Boundary: Lower limit of band 3; Expected bonus band: 10%
- **B6** – Years: 11; Boundary: Just inside band 3; Expected bonus band: 10%
- **B7** – Years: 14; Boundary: Upper limit of band 3; Expected bonus band: 10%
- **B8** – Years: 15; Boundary: Lower limit of band 4; Expected bonus band: 15%

- **B9** – Years: 16; Boundary: Just inside band 4; Expected bonus band: 15%
- **B10** – Years: 0 and -1; Boundary: Minimum and below minimum; Expected bonus band: 0% and error

Each of these tests is combined with the lowest, a middle and the highest rating (1, 3, 5) so that the bonus amount is verified at the boundaries of both inputs. The expected amount is calculated independently, for example bonus = salary × band percentage × rating factor, and compared with the system result.

iv) Why ECP and BVA are better than exploratory testing here

- **Precision:** the logic is a set of exact numeric thresholds. BVA targets exactly those thresholds, whereas exploratory testing depends on the tester's intuition and may never try exactly 5, 10 or 15.
- **Repeatability:** structured test cases can be re-run after every correction and every release, which is essential for payroll rules that change yearly.
- **Measurable coverage:** the team can show that all classes and all boundaries were tested, which is needed for financial audits.
- **Efficiency:** about 10 boundary cases replace hundreds of random employee records.
- **Risk:** an error in payroll affects salaries and legal compliance, so uncertainty about coverage is not acceptable.

Exploratory testing still has a place for discovering unexpected scenarios, such as mid-year joiners, leave without pay or retroactive changes, but it should come after the formal ECP and BVA suite.

v) Conclusion

Redesigning the suite with ECP and BVA, adding independent calculation of expected values, and including these tests in the automated regression suite would have detected the incorrect payouts at 5, 10 and 15 years before release and will prevent a repeat.

7. A laboratory temperature control system with complex if-else logic failed to raise the alarm after a particular sensor-malfunction sequence. Construct a CFG, evaluate branch coverage gaps, and propose a white-box strategy achieving 100% MC/DC.

Answer:

In a safety-critical controller, each combination of conditions that can lead to an alarm must be verified. The alarm was missed because the testing satisfied branch coverage without checking that every individual condition independently affects the alarm decision.

i) Conceptual control logic

- N1: Read temperature T, sensor status and alarm-silence flag.
- D1: If T < Tmin, switch the heater on (N2).

- D2: Else if T > Tmax, switch the cooler on (N3); otherwise remain idle (N4).
- D3: If (sensorFault OR T > Tcrit) AND NOT silenced, raise the alarm (N5); otherwise skip (N6).
- N7: Log status and end.

Let A = sensorFault, B = (T > Tcrit) and C = silenced. The alarm decision is D = (A OR B) AND NOT C.

ii) Control flow graph summary

The graph has three decision nodes (D1, D2, D3), so V(G) = 3 + 1 = 4 at the decision level. The alarm decision D3 additionally contains three conditions (A, B, C), which a simple branch view treats as a single true/false outcome.

iii) Evaluation of branch coverage gaps

A typical test set contains: normal temperature, low temperature, high temperature, and critical temperature. These tests make D1, D2 and D3 each evaluate to both true and false, so branch coverage reaches 100%. But in none of these tests is the sensor fault combined with the silence flag. The scenario that failed in production was a sensor malfunction while the alarm silence flag was left set from an earlier event, so the decision depended on condition C in a combination that was never tried. Branch coverage only checks that the whole decision was true or false, not that each condition can independently change the result.

iv) MC/DC test design

Modified Condition/Decision Coverage requires that every condition is shown to independently change the decision outcome. For three conditions, a minimum of n + 1 = 4 tests are needed.

- **M1** – A (sensorFault): True; B (T > Tcrit): False; C (silenced): False; Decision D: True (alarm); Shows independence of: Base for A and C
- **M2** – A (sensorFault): False; B (T > Tcrit): False; C (silenced): False; Decision D: False; Shows independence of: A (compare M1 and M2)
- **M3** – A (sensorFault): False; B (T > Tcrit): True; C (silenced): False; Decision D: True; Shows independence of: B (compare M2 and M3)
- **M4** – A (sensorFault): True; B (T > Tcrit): False; C (silenced): True; Decision D: False; Shows independence of: C (compare M1 and M4)

Verification: M1 and M2 differ only in A and give different outputs; M2 and M3 differ only in B and give different outputs; M1 and M4 differ only in C and give different outputs. M4 is the combination, sensor fault with alarm silenced, that failed in production, and it now shows whether the design suppresses a sensor-fault alarm.

v) Overall white-box strategy

- Create one test for each of the four basis paths to achieve 100% statement and branch coverage.
- Add the four MC/DC tests above for the alarm decision.
- Add boundary tests at T = Tmin − 1, Tmin, Tmax, Tmax + 1, Tcrit − 1, Tcrit and Tcrit + 1.
- Add sequence tests (sensor failing, recovering, failing again) and fault-injection tests for stuck-at and out-of-range sensor values.
- Use a coverage tool that reports MC/DC and review any uncovered condition manually.

vi) Impact on safety-critical reliability

MC/DC is the criterion required for the highest software levels in standards such as DO-178C, because it exposes faults hidden in compound conditions with a manageable number of tests (n + 1 instead of 2^n). Applying it ensures that the alarm cannot be suppressed by a condition combination that testers did not examine. The result is a higher level of confidence that the laboratory system will react correctly to dangerous situations.

8. A travel booking portal allows several promo codes, but some combinations produce a negative total amount. Analyse the limits of isolated input testing, apply decision table or cause-effect testing, and propose an optimised test set.

Answer:

When promo type, price and user tier are tested individually, each field behaves correctly, but the defect appears only when a percentage promo, a flat promo and a tier discount are applied together to a low price. Isolated testing cannot find such failures because the error is in the interaction among the inputs.

i) Limitations of isolated testing

- It verifies each discount rule alone and therefore never produces a combination where the total discount exceeds the price.
- It misses ordering effects, for example applying the flat discount before or after the percentage discount.
- It does not test the same code applied twice or two exclusive codes applied together.
- Exhaustive combination testing is impractical as the number of codes and tiers grows.

ii) Assumed business rules

- A booking can have at most one percentage promo (for example 20%) and one flat promo (for example ₹1,000).
- Gold-tier users receive an additional 5% discount.
- The combined percentage discounts (promo + tier) cannot exceed 50% of the base price, and the final amount after all discounts can never be below ₹0.

iii) Decision table

Conditions: C1 = percentage promo applied, C2 = flat promo applied, C3 = Gold tier.

- **R1** – C1 %: N; C2 Flat: N; C3 Gold: N; Expected outcome: No discount; total = price
- **R2** – C1 %: N; C2 Flat: N; C3 Gold: Y; Expected outcome: 5% tier discount only
- **R3** – C1 %: Y; C2 Flat: N; C3 Gold: N; Expected outcome: Percentage promo only
- **R4** – C1 %: Y; C2 Flat: N; C3 Gold: Y; Expected outcome: Percentage + tier; discount capped at 50%
- **R5** – C1 %: N; C2 Flat: Y; C3 Gold: N; Expected outcome: Flat discount only; total not below ₹0
- **R6** – C1 %: N; C2 Flat: Y; C3 Gold: Y; Expected outcome: Flat + tier; total not below ₹0
- **R7** – C1 %: Y; C2 Flat: Y; C3 Gold: N; Expected outcome: Both promos; cap at 50%; total not below ₹0
- **R8** – C1 %: Y; C2 Flat: Y; C3 Gold: Y; Expected outcome: All three; cap at 50%; total not below ₹0

iv) Cause-effect graph view

Causes are the three conditions above plus the price. Effects are E1 = apply discounts, E2 = cap at 50%, E3 = floor total at ₹0, and E4 = reject the duplicate or invalid code. The graph shows that E2 depends on C1 AND (C2 OR C3), and E3 depends on C2. This helps identify which rules can really cause the negative total, namely every rule containing C2 combined with a low price.

v) Optimised test set

- **T1** – Rule: R1; Price (₹): 5,000; Codes / tier: None / Regular; Expected result: 5,000
- **T2** – Rule: R2; Price (₹): 5,000; Codes / tier: None / Gold; Expected result: 4,750
- **T3** – Rule: R3; Price (₹): 5,000; Codes / tier: 20% / Regular; Expected result: 4,000
- **T4** – Rule: R4; Price (₹): 5,000; Codes / tier: 20% / Gold; Expected result: 3,750
- **T5** – Rule: R5; Price (₹): 5,000; Codes / tier: ₹1,000 flat / Regular; Expected result: 4,000
- **T6** – Rule: R7; Price (₹): 5,000; Codes / tier: 20% + ₹1,000 / Regular; Expected result: 3,000
- **T7** – Rule: R8; Price (₹): 5,000; Codes / tier: 20% + ₹1,000 / Gold; Expected result: 2,750
- **T8** – Rule: R5 boundary; Price (₹): 800; Codes / tier: ₹1,000 flat / Regular; Expected result: 0 (not negative)
- **T9** – Rule: R7 boundary; Price (₹): 1,000; Codes / tier: 20% + ₹1,000 flat / Regular; Expected result: 0 (not negative)

- **T10** – Rule: R8 boundary; Price (₹): 1,000; Codes / tier: 20% + ₹1,000 / Gold; Expected result: 0 (not negative)
- **T11** – Rule: Invalid; Price (₹): 5,000; Codes / tier: Same flat code twice; Expected result: Second code rejected
- **T12** – Rule: Invalid; Price (₹): 0 / −100; Codes / tier: Any code; Expected result: Price error

vi) Justification

The decision table gives complete coverage of all eight combinations with only twelve tests, instead of hundreds of random combinations. Boundary tests T8 to T10 target the exact defect, a discount larger than the price. By considering rules that behave identically, redundant tests are removed, which maximises defect detection per test. The same table can also be turned into automated parameterised tests and kept as a regression suite.

9. A legacy database migration tool with deeply nested loops shows high statement coverage but frequent infinite-loop crashes. Evaluate the white-box approach, explain why statement coverage was insufficient, and propose path testing with loop boundaries using cyclomatic complexity.

Answer:

High statement coverage means that every line of code was executed at least once, but it says nothing about how many times a loop runs, in which order decisions are taken, or whether a loop can ever terminate. Infinite loops are therefore not detected by statement coverage, which explains why the tool passed testing and then crashed in production.

i) Conceptual structure of the module

- 1: for each record i in the input file (outer loop, D1)
- 2: for each field j in record i (inner loop, D2)
- 3: if field is empty, apply default value (D3)
- 4: while (field is not valid) → repair(field) (D4)
- 5: mark record as validated
- 6: return migration report

ii) Why statement coverage was insufficient

- A single record whose field becomes valid after one repair executes every statement, so coverage is 100%, yet the case where repair() can never make the field valid is not tested. In that case the while loop at line 4 never terminates.
- Statement coverage does not require the zero-iteration case (an empty file or record with no fields), which can cause null-pointer or empty-report errors.

- It does not require testing the combinations of loop counts in nested loops, such as outer loop many times and inner loop zero times.
- It does not test decision outcomes separately; for example, the case where the field is not empty (false branch of D3) may never be executed.

iii) Cyclomatic complexity

The predicate nodes are D1, D2, D3 and D4, so V(G) = 4 + 1 = 5. This is the minimum number of independent paths that must be executed, and a measure of the testing effort and of the risk, since values above 10 are normally considered hard to test and maintain.

- **P1** – Description: Outer loop not entered; Test input: Empty file
- **P2** – Description: Outer loop once, inner loop not entered; Test input: One record with no fields
- **P3** – Description: Inner loop entered, field empty (default applied), already valid; Test input: One record, one empty field
- **P4** – Description: Inner loop entered, field not empty and valid (D3 false, D4 false); Test input: One record, one valid field
- **P5** – Description: Field invalid, while loop entered and repaired; Test input: One record, one repairable field

iv) Loop boundary testing

- **Outer loop (records)** – Tests: 0, 1, 2, a typical number (for example 100), maximum supported records, maximum + 1
- **Inner loop (fields)** – Tests: 0, 1, 2, typical number, maximum fields, maximum + 1 (tested with the outer loop at its minimum)
- **Nested loops** – Tests: Test the innermost loop first with outer loops at minimum values, then work outward, with the outer loops at typical values
- **While (repair) loop** – Tests: Repaired at the first attempt, repaired after 2 attempts, repaired at the maximum attempts, never repairable (must stop or report an error)

v) Proposed strategy

- Redraw the CFG including every loop-back edge and loop exit.
- Execute all five basis paths and verify 100% branch coverage, not only statement coverage.
- Apply the loop-boundary tests above, with a timeout so that a non-terminating test fails instead of hanging.
- Add a fault-injection test in which repair() returns an unrepairable value, and fix the code with a retry limit (for example 3 attempts) and an error log.

- Use static analysis and a complexity tool to flag modules whose V(G) exceeds the threshold, and refactor deep nesting.

vi) Justification

Cyclomatic complexity gives an objective lower bound of five independent paths, and loop-boundary testing addresses the situations where defects in loops occur: zero, one, many and too many iterations. Together they detect non-terminating loops and off-by-one errors that statement coverage cannot see, which improves the stability of the migration tool and prevents production crashes.

10. A food delivery app's Real-time Tracker moves through states (Ordered, Preparing, Out for Delivery, Delivered) but gets stuck when a driver cancels mid-delivery. Evaluate the black-box strategy and apply State Transition Testing.

Answer:

The tracker is a stateful workflow: the result of an event depends on the current state. ECP and BVA treat inputs independently and ignore history, so they cannot represent a situation such as a driver cancelling after the order is already out for delivery. The existing tests probably covered only the normal path from Ordered to Delivered.

i) Evaluation of the current strategy

- Only the happy path was tested; unusual events such as cancellation, a driver becoming unavailable or the restaurant rejecting the order were not modelled.
- The state model had no state for the situation after the driver cancels, so the application could not move forward.
- Invalid transitions (for example Delivered back to Preparing) were not verified to be rejected.

ii) State model

States: Ordered (S1), Preparing (S2), Out for Delivery (S3), Delivered (S4), Reassigning Driver (S5), Cancelled (S6). Events: restaurant accepts, food ready and driver picks up, delivery confirmed, driver cancels, new driver assigned, customer cancels, no driver found within the timeout.

- **Ordered** – Event: Restaurant accepts; Next state: Preparing; Valid?: Valid
- **Ordered** – Event: Customer cancels; Next state: Cancelled; Valid?: Valid
- **Preparing** – Event: Driver picks up food; Next state: Out for Delivery; Valid?: Valid
- **Preparing** – Event: Customer cancels; Next state: Cancelled (refund as per policy); Valid?: Valid
- **Out for Delivery** – Event: Delivery confirmed; Next state: Delivered; Valid?: Valid
- **Out for Delivery** – Event: Driver cancels; Next state: Reassigning Driver; Valid?: Valid (the defect area)

- **Reassigning Driver** – Event: New driver assigned; Next state: Out for Delivery; Valid?: Valid
- **Reassigning Driver** – Event: No driver within timeout; Next state: Cancelled (full refund); Valid?: Valid
- **Delivered** – Event: Any delivery event; Next state: Delivered (ignored); Valid?: Invalid, must be rejected
- **Cancelled** – Event: Driver picks up food; Next state: Cancelled; Valid?: Invalid, must be rejected

iii) Test scenarios

- **ST1** – Sequence: Ordered → Preparing → Out for Delivery → Delivered; Expected result: Normal flow, customer notified at each step
- **ST2** – Sequence: Out for Delivery → driver cancels; Expected result: State changes to Reassigning Driver, customer is informed, tracker does not freeze
- **ST3** – Sequence: Reassigning Driver → new driver assigned → Delivered; Expected result: Tracker resumes and completes
- **ST4** – Sequence: Reassigning Driver → timeout without a driver; Expected result: Order cancelled and refund triggered
- **ST5** – Sequence: Ordered → customer cancels; Expected result: Cancelled, restaurant notified
- **ST6** – Sequence: Delivered → driver cancels (invalid); Expected result: Event ignored, state remains Delivered
- **ST7** – Sequence: Driver cancels twice in a row; Expected result: Handled without duplicate refunds or reassignments
- **ST8** – Sequence: App closed and reopened during Reassigning Driver; Expected result: Correct state restored from the server

iv) Coverage criteria

- All-states coverage: every state is visited at least once (ST1 to ST5).
- All-transitions (0-switch) coverage: each valid transition in the table is executed at least once.
- 1-switch coverage: sequences of two transitions, such as driver cancels followed by new driver assigned, are executed.
- Negative coverage: invalid state-event pairs are tried and must not change the state.

v) Why this technique finds what ECP and BVA miss

ECP and BVA select values of an input field, but the failure here does not depend on a value; it depends on the order of events and the state in which the event arrives. A cancellation is

valid in Ordered and in Out for Delivery but leads to different outcomes. State Transition Testing explicitly models this dependency, makes missing states and transitions visible (the missing Reassigning Driver state), and tests invalid transitions and sequences. The redesigned scenarios therefore prevent the tracker from getting stuck and make dynamic workflows reliable.

11. A multi-factor authentication module (password strength, OTP timing, IP location) has brute-force vulnerabilities despite testing. Assess the black-box and white-box integration and propose a hybrid strategy with Error Guessing and formal Path Analysis.

Answer:

The vulnerability shows that the module was tested for correct behaviour (valid and invalid credentials) but not for hostile behaviour. Functional black-box tests rarely reach the paths where attackers repeat requests, mix conditions or bypass a step, and white-box tests written for coverage seldom ask whether each path enforces the security policy.

i) Assessment of the existing approach

- Black-box tests covered valid and invalid passwords, an expired OTP, and a blocked IP, but each factor was tested separately and with one attempt only.
- White-box tests aimed at code coverage; statements were executed, but the order and counting of failed attempts was not examined.
- Security properties such as rate limiting, account lockout and OTP single-use were not defined as test objectives.
- Error guessing, which uses the tester's experience of typical attacks, was not applied systematically.

ii) Formal path analysis

A simplified authentication flow is: (1) check the lockout status → (2) verify the password → (3) verify the OTP and its time window → (4) check the IP location rule → (5) issue the session token. Decisions are lockout (D1), password valid (D2), OTP valid and in time (D3), IP allowed (D4). V(G) = 4 + 1 = 5.

- **P1** – Route: D1 true → reject; Security question to verify: Locked accounts stay locked and the lock cannot be bypassed by changing IP or device
- **P2** – Route: D1 false, D2 false → reject; Security question to verify: Failed passwords are counted and trigger lockout after the limit
- **P3** – Route: D2 true, D3 false → reject; Security question to verify: Failed OTP attempts are also counted (a common defect: counter is reset or not incremented here)
- **P4** – Route: D2 true, D3 true, D4 false → reject or step-up; Security question to verify: Unfamiliar IP requires extra verification and is logged

- **P5** – Route: All true → token; Security question to verify: Token is issued only after all factors; OTP cannot be reused

Path analysis can expose a typical flaw: the attempt counter is incremented only on password failure, so an attacker with a known password can try all one million six-digit OTPs without being locked out (path P3). Coverage tests would not notice this, because P3 executes without error.

iii) Error guessing test cases

- Repeated wrong passwords (for example 5, 10, 100 attempts) to check lockout and delays.
- Brute-forcing the OTP: many attempts in sequence and in parallel, to find missing rate limits and race conditions.
- Reusing an already-used OTP, and using an OTP just before and just after expiry (29, 30, 31 seconds if the window is 30 seconds).
- Changing or spoofing the IP address with header values such as X-Forwarded-For after a lockout.
- SQL-injection and special-character input in username and password, very long passwords, Unicode and case variations.
- Skipping a step by calling the OTP or token API directly; replaying an old session token.
- Weak passwords at the boundary of the strength rule (7 and 8 characters, no digit, common passwords).
- Resetting the lockout by restarting the app, clearing cookies or switching devices.

iv) Hybrid strategy

- Define security requirements (for example lock after 5 failures; OTP valid 30 seconds and once; limit 10 requests per minute per IP) so that every test has an expected result.
- Use path analysis to derive one test per basis path, then use error guessing to attack each path with hostile inputs.
- Use ECP and BVA on password length, OTP age and attempt counts (4, 5 and 6 attempts around the limit of 5).
- Add automated security tools (OWASP ZAP, Burp Suite, a load tool for concurrent attempts) and static analysis (SAST).
- Log and review all authentication failures; add every discovered vulnerability to the regression suite.

v) How this improves the security posture

Formal path analysis guarantees that every route through the authentication code is examined, and error guessing adds attacker thinking that no specification contains. The integrated

approach detects flaws in the counting, timing and sequencing of factors, which are the usual causes of brute-force vulnerabilities, and gives measurable evidence that lockout, OTP and IP rules are enforced on every path.

12. An e-learning platform with tiered access (Free, Basic, Premium) based on subscription dates shows errors when a user upgrades exactly on the expiry date. Evaluate the testing criteria and apply ECP to date ranges and subscription types.

Answer:

The error on the expiry day shows that the date boundary and the order of operations (expire first or upgrade first) were not specified or tested. The current test design probably used dates well before or well after the expiry date, so the transition day was never exercised.

i) Evaluation of the current testing criteria

- The criteria did not include date boundaries such as expiry date minus 1, expiry date, and expiry date plus 1.
- Only simple state tests (active user, expired user) were designed; transitions between tiers on specific days were not.
- Time details such as the end of the day (23:59:59), time zone and the server clock were not considered.
- Combinations of the old tier, new tier and course type were not covered together.

ii) Equivalence classes for subscription date

- **D1** – Date condition: More than 1 day before expiry; Representative: Expiry − 10 days; Expected access: Full access to the current tier
- **D2** – Date condition: Day before expiry; Representative: Expiry − 1 day; Expected access: Full access to the current tier
- **D3** – Date condition: On the expiry date; Representative: Expiry day; Expected access: Full access until the end of the day (as defined); upgrade accepted with no gap
- **D4** – Date condition: Within the grace period after expiry; Representative: Expiry + 1 day; Expected access: Per policy: limited access or renewal prompt
- **D5** – Date condition: Beyond the grace period; Representative: Expiry + 30 days; Expected access: Reverts to Free tier
- **D6 (invalid)** – Date condition: Invalid date or future start; Representative: 31-02-2026, start in future; Expected access: Error message

iii) Equivalence classes for subscription type and course type

- **Free** – Course type: Free course; Expected access: Allowed
- **Free** – Course type: Basic or Premium course; Expected access: Blocked, upgrade prompt

- **Basic** – Course type: Free or Basic course; Expected access: Allowed
- **Basic** – Course type: Premium course; Expected access: Blocked, upgrade prompt
- **Premium** – Course type: All courses; Expected access: Allowed
- **Expired Basic or Premium** – Course type: Any paid course; Expected access: Blocked after grace period

iv) Structured test design

- **E1** – Current tier / date class: Basic / D1; Action: Access Basic course; Expected result: Allowed
- **E2** – Current tier / date class: Basic / D2; Action: Upgrade to Premium; Expected result: Premium active immediately, remaining period handled as per policy
- **E3** – Current tier / date class: Basic / D3 (expiry day, 10:00); Action: Upgrade to Premium; Expected result: Upgrade succeeds, no loss of access
- **E4** – Current tier / date class: Basic / D3 (23:59:59); Action: Upgrade to Premium; Expected result: Upgrade succeeds, new period starts on the next day
- **E5** – Current tier / date class: Basic / D3; Action: Do nothing until 00:00:01 next day; Expected result: Moves to D4 behaviour
- **E6** – Current tier / date class: Basic / D4; Action: Upgrade to Premium; Expected result: Premium activated, new dates calculated from the upgrade date
- **E7** – Current tier / date class: Basic / D5; Action: Access Basic course; Expected result: Blocked, Free tier
- **E8** – Current tier / date class: Free / any date; Action: Upgrade to Basic and then to Premium; Expected result: Each tier activated correctly
- **E9** – Current tier / date class: Premium / D3; Action: Downgrade to Basic; Expected result: Downgrade effective at the end of the paid period
- **E10** – Current tier / date class: Premium / D3; Action: Renew same tier; Expected result: Period extended without gap or duplicate payment
- **E11** – Current tier / date class: Any; Action: Upgrade with invalid date or in another time zone; Expected result: Error or correct conversion

v) Justification of completeness

- Every date class and every tier and course combination is covered at least once.
- The expiry date boundary is tested with the day before, the day itself (at start and end of day) and the day after, which are the values where the reported defect occurs.
- Upgrade, downgrade, renewal and no-action paths are all covered, so that transitions among Free, Basic and Premium are verified in both directions.

- Representative values avoid hundreds of date combinations while keeping coverage of every behaviour class.

With this design, users upgrading on the day of expiry are treated consistently, access is never lost or duplicated, and the tests can be automated and re-run whenever pricing or subscription rules change.

13. A smart-home lighting controller uses switch-case statements and complex Boolean expressions. Evaluate the risks of statement coverage alone, apply Branch and Condition Coverage, and design an improved white-box strategy.

Answer:

Statement coverage only guarantees that every line was executed once. In code with switch-case statements and compound Boolean expressions, it can reach 100% while important outcomes and conditions are never evaluated, so failures appear only when real sensors and lights are connected.

i) Conceptual code

- switch (mode): case DAY, case NIGHT, case AUTO, default.
- In AUTO mode: if (lux < 30 && (motion || manualOverride)) then dim lights to 70%, else if (lux < 80) then dim to 40%, else turn lights off.

Let A = (lux < 30), B = motion detected, C = manualOverride. The decision is D = A AND (B OR C).

ii) Risks of relying only on statement coverage

- A single test with lux = 20 and motion = true executes the statements of the dimming branch, but the else branch and the other conditions are not exercised.
- Missing else or default branches are not detected, because no statement exists to be covered; an unexpected mode value may leave the lights in an undefined state.
- A switch case with a missing break statement may fall through to the next case without any statement being skipped.
- Wrong operators (AND instead of OR) or a wrong threshold can remain hidden because the statements still execute.
- The risk is highest in hardware-software integration, where sensor noise and unexpected values make rarely used branches run.

iii) Branch coverage

Branch coverage requires every decision to evaluate to both true and false, and every case of the switch to be taken. Tests are needed for DAY, NIGHT, AUTO and default; D true and D false; lux < 80 true and false.

- **BR1** – Mode: DAY; lux: any; motion: any; manual: any; Expected outcome: Day behaviour

- **BR2** – Mode: NIGHT; lux: any; motion: any; manual: any; Expected outcome: Night behaviour
- **BR3** – Mode: AUTO; lux: 20; motion: True; manual: False; Expected outcome: Dim 70% (D true)
- **BR4** – Mode: AUTO; lux: 50; motion: False; manual: False; Expected outcome: Dim 40% (D false, lux < 80)
- **BR5** – Mode: AUTO; lux: 120; motion: False; manual: False; Expected outcome: Lights off
- **BR6** – Mode: Invalid value; lux: any; motion: any; manual: any; Expected outcome: Default case: safe state and error log

iv) Condition coverage and MC/DC

Condition coverage requires each atomic condition (A, B, C) to be true and false at least once. A stronger set, which shows that each condition independently influences the outcome, is:

- **C1** – A (lux<30): True; B (motion): True; C (manual): False; D: True; Shows: Base case
- **C2** – A (lux<30): False; B (motion): True; C (manual): False; D: False; Shows: A changes the outcome
- **C3** – A (lux<30): True; B (motion): False; C (manual): False; D: False; Shows: B changes the outcome
- **C4** – A (lux<30): True; B (motion): False; C (manual): True; D: True; Shows: C changes the outcome

Weak condition coverage alone could be satisfied by two tests (T,T,F) and (F,F,T), but both tests would still pass if the code ignored manualOverride completely (A AND B). Test C4 (expected true, faulty code gives false) reveals that defect, which is why the four MC/DC-style tests are stronger.

v) Improved white-box strategy

- Measure statement, branch and condition coverage with a tool and require 100% branch and condition coverage for the controller logic.
- Test every switch case and the default, and review code for missing break statements.
- Add boundary tests at the thresholds: lux = 29, 30, 31 and 79, 80, 81.
- Use sensor simulators for noise, stuck values, negative or very large readings, and sudden changes.
- Add hardware-in-the-loop tests for timing, communication delays and the dimming driver.

vi) Reduction of hardware-software integration risk

The extra tests check that the software handles every combination of sensor conditions and reacts safely to unexpected values before it is connected to the real hardware. This removes logic errors in advance, so integration problems are limited to genuine hardware behaviour, which shortens debugging and increases the reliability of the lighting system.

14. A logistics system fails to detect errors when weights are zero or negative. Analyse the input validation testing, apply Boundary Value Analysis (robustness testing) with invalid boundaries, and justify negative scenario testing.

Answer:

Shipping cost is calculated from weight, destination zone and delivery speed. The system accepted zero and negative weights, so shipping was calculated as zero or even negative. The testing only used valid weights; invalid input behaviour was never specified as a test objective.

i) Analysis of the failure

- The requirement did not clearly state the valid range, so the tests and the code had no common reference.
- Testing used positive typical values such as 2 kg, 10 kg and 25 kg, which all pass.
- Validation was probably present only in the user interface; the API and database accepted any number.
- No negative scenario was planned, so a missing check on the lower limit was undetected.

ii) Assumed rule and robustness BVA

Assume valid weight is 1 kg to 30 kg. Standard BVA tests min−, min, min+, nominal, max−, max, max+. Robustness testing explicitly includes the values just outside the range and other invalid inputs.

- **W1** – Weight input: -1; Type: Negative (invalid); Expected result: Error: weight must be at least 1 kg
- **W2** – Weight input: 0; Type: Just below minimum; Expected result: Error: weight must be at least 1 kg
- **W3** – Weight input: 1; Type: Minimum; Expected result: Accepted, cost calculated
- **W4** – Weight input: 2; Type: Just above minimum; Expected result: Accepted
- **W5** – Weight input: 15; Type: Nominal; Expected result: Accepted
- **W6** – Weight input: 29; Type: Just below maximum; Expected result: Accepted
- **W7** – Weight input: 30; Type: Maximum; Expected result: Accepted
- **W8** – Weight input: 31; Type: Just above maximum; Expected result: Error or special-freight handling

- **W9** – Weight input: 0.5 / 1.001; Type: Decimal values; Expected result: Handled according to the rounding rule
- **W10** – Weight input: blank, abc, 1e9, 999999999; Type: Missing, text, extreme; Expected result: Error message, no crash

The same robustness idea is applied to the other inputs: destination zone (valid zones 1 to 5; test 0, 1, 5, 6 and blank) and delivery speed (Standard, Express, Overnight; test an unknown value). Then valid boundary weights are combined with each zone and speed using a decision table or pairwise selection, so that cost calculations are verified for each combination.

iii) Why negative scenarios are necessary

- **Financial protection:** a zero or negative weight produces a zero or negative shipping price, which is direct revenue loss and a possible fraud route.
- **Data integrity:** invalid values saved in the database corrupt reports, invoices and route planning.
- **Robustness:** real users and other systems send incorrect, empty or malicious data; a system that has only been tested with correct data cannot be assumed stable.
- **Security:** unvalidated input is the basis of injection and overflow attacks, so negative tests are also a first security check.
- **User experience:** clear error messages guide users to correct their input, instead of showing crashes or wrong prices.
- **Cost of defects:** finding the validation gap in test is far cheaper than correcting invoices after shipment.

iv) Recommended improvements

- Define valid ranges and error messages in the requirement specification.
- Validate at every layer: interface, API and database constraints.
- Automate the robustness tests W1 to W10 and run them with every build.
- Add every defect found in production as a regression test.

In conclusion, robustness BVA ensures that the system not only calculates correctly for valid shipments but also rejects invalid input safely, which is essential for a stable and trustworthy logistics system.

15. A medical imaging application using parallel processing loops gives inconsistent results only when the data size is a prime number. Evaluate reliance on random testing and propose a structured white-box approach with Basis Path Testing.

Answer:

Results that differ only for prime data sizes suggest that the code divides the data among parallel workers and handles the remainder incorrectly. A prime size cannot be divided

evenly by any number of threads greater than one, so a leftover part always exists. Random testing rarely selects such sizes and, in a mathematical module, cannot give confidence.

i) Evaluation of random testing

- Most random sizes (powers of two, multiples of the thread count) divide evenly, so the remainder logic is never executed.
- Failures are intermittent and depend on the number of threads, so a random failure cannot be reproduced.
- There is no coverage measure; the team cannot say whether the remainder path or the small-size path was tested.
- For imaging, a wrong result may not crash the program but silently affects diagnosis, so unnoticed errors are not acceptable.

ii) Conceptual parallel partition code

- 1: if ($n \leq 0$) return error (D1)
- 2: chunk = n / T (integer division, T = number of threads)
- 3: if ($n < T$) use fewer threads (D2)
- 4: for each thread t from 0 to $T-1$ (D3): start = $t \times$ chunk, end = start + chunk
- 5: if (t is the last thread) end = n (D4), which gives the last thread the remainder
- 6: process(start, end) in parallel and merge the results

A typical defect is that the remainder statement at line 5 is missing or wrong: with n = 101 and T = 4, chunk = 25 and only 100 items are processed, so the last element is ignored. When n is divisible by T the defect never appears.

iii) Basis path testing

The decisions D1 to D4 give V(G) = 4 + 1 = 5 independent paths.

- **P1** – Condition: $n \leq 0$; Test (T = 4): n = 0; Expected result: Error returned, no processing
- **P2** – Condition: $n < T$; Test (T = 4): n = 3; Expected result: Handled with fewer threads, all items processed
- **P3** – Condition: $n \geq T$, divisible, loop runs, last thread without remainder; Test (T = 4): n = 100; Expected result: Result equals sequential result
- **P4** – Condition: $n \geq T$, loop runs, last thread takes remainder; Test (T = 4): n = 101 (prime); Expected result: All 101 items processed
- **P5** – Condition: Loop boundary: one thread (T = 1); Test (T = 4): n = 97, T = 1; Expected result: Same as sequential result

iv) Additional structured tests

- Boundary and prime sizes: 1, 2, 3, 5, 7, 13, 97, 101, 9973, together with sizes one above and below multiples of the thread count (T−1, T, T+1, 2T−1, 2T+1).
- Reference oracle: compare the parallel result with a trusted sequential computation, using a tolerance for floating-point rounding (for example 1e-9) because parallel summation order changes rounding.
- Different thread counts (1, 2, 4, 8, 16) with the same data, which must all give the same result.
- Concurrency checks with tools such as ThreadSanitizer or Helgrind for race conditions and with stress runs repeated many times.
- Data-flow tests for the shared variables used in the merge step (definition and use of each result array).
- Invariants: sum of all chunk sizes must equal n; no item processed twice; no item skipped.

v) How this ensures mathematical accuracy and stability

Basis path testing forces the remainder path (P4) and the small-size path (P2) to be executed, which random sizes rarely do. Prime and boundary sizes check the exact condition that triggers the defect, and the sequential oracle detects any numerical difference. Race detection and thread-count variation confirm that the result does not depend on scheduling. After the correction, the tests are automated and run on every build, so the imaging software gives the same accurate results for any data size, and clinicians can rely on its output.

RUBRICS FOR CALCULATION:

Problem Understanding & Context Analysis (25) – Level 4 (Exemplary): Thoroughly understands the scenario with accurate interpretation of all requirements; Level 3 (Proficient): Clearly understands the scenario with minor gaps; Level 2 (Developing): Basic understanding; some aspects of the scenario unclear; Level 1 (Beginning): Poor understanding or misinterpretation of the scenario.

Application of Concepts (30) – Level 4 (Exemplary): Applies appropriate concepts effectively to solve complex real-world problems; Level 3 (Proficient): Applies relevant concepts correctly with minor errors; Level 2 (Developing): Applies basic concepts but with limited accuracy; Level 1 (Beginning): Fails to apply appropriate concepts.

Analytical & Decision-Making Skills (20) – Level 4 (Exemplary): Demonstrates strong analysis and selects optimal solutions with justification; Level 3 (Proficient): Good analysis with reasonable solutions; Level 2 (Developing): Limited analysis; solutions lack justification; Level 1 (Beginning): Poor or no analysis; incorrect decisions.

Solution Design & Approach (15) – Level 4 (Exemplary): Well-structured, innovative, and feasible solution approach; Level 3 (Proficient): Logical and structured solution with minor gaps; Level 2 (Developing): Basic solution with limited structure or feasibility; Level 1 (Beginning): Unclear or incorrect solution approach.

Clarity, Justification & Presentation (10) – Level 4 (Exemplary): Clear, well-justified explanation with strong reasoning; Level 3 (Proficient): Clear explanation with some justification; Level 2 (Developing): Limited clarity and weak justification; Level 1 (Beginning): Poorly explained with no justification.